

Three lenses, one mess

Logic programming for the rest of us

Burin Choomnuan · FP Syd · 2026

whoami

Burin Choomnuan · Sydney · github.com/burinc

- Clojure & functional-programming developer — and a logic-programming tinkerer
- I build developer tooling: static analysis, MCP servers, ClojureDart mobile apps
- Author of **spock** — the toolkit in this talk: Prolog + SMT + call-graph, shipped as a Babashka CLI and a ~50ms GraalVM native binary

Three solver-shaped questions I keep hitting in real code — and the tools that fit them.

A bug you've probably shipped

```
(defn can-delete? [user]
  ;; alice has :admin role
  (and (has-role? user :admin)
        ;; ...but we explicitly denied alice
        (not= user :alice)))
```

Three rules in three places: assignment, grant, deny. They contradict.

How would you find out — *before* prod?

The thesis

Logic programming has the right *shape* for three kinds of question:

- **Relations** — who can do what?
- **Conflicts** — is this consistent?
- **Reachability** — can this state happen?

Three flavours. One talk. One demo.

Imperative or relational?

```
:: The imperative answer (~ 8 lines, plus a recursion you write yourself)
(defn users-who-can [permission]
  (->> (all-users)
    (filter #(roles-permit? (roles-of %) permission))
    (mapcat resolve-role-inheritance)
    distinct))

:: The relational answer (one query)
(tu/solve {} '(can :user delete))
```

The relational version doesn't *traverse* — it *describes*.

Spock's DSL — Clojure data is Prolog

Clojure	Prolog
symbol	atom
keyword	variable
vector	list

```
'[(parent [father son])
 (parent [son grandson])
 (ancestor [:x :y] [(parent :x :y)])
 (ancestor [:x :y] [(parent :x :z) (ancestor :z :y)])]
```

No macros. No reader literals. Just a vector of vectors.

RBAC, in Prolog

```
(def rbac-rules
  '[(has-role [alice admin])
    (parent-role [admin user])
    (role-permits [admin delete])
    (role-permits [user read])

    (can [:u :p]
      [(has-role :u :r)
       (role-permits :r :p)])

    (can [:u :p]
      [(has-role :u :r)
       (parent-role :r :pr)
       (role-permits :pr :p)])])
```

Eight clauses. The recursion in the second `can` is what gives us role inheritance.

Querying — same shape as `map`

```
(def s (tu/solver rbac-rules))

(tu/solve {:rules s} '(can :user delete))
;; => [{:user alice}]

(tu/solve {:rules s} '(can :user read))
;; => [{:user alice}] ; via inheritance!

(tu/solve-n 1 {:rules s} '(can alice :perm))
;; => [{:perm delete}]
```

`solve` returns a seq of substitution maps. Composes with everything.

Theories are immutable values

```
(require '[spock.theory :as theory])

(def base (theory/parse-theory "parent(a,b). parent(b,c)."))
(def extra (theory/parse-theory "parent(c,d)."))

(def merged (theory/merge-theories base extra))
(theory/theory-size merged) ;; => 3
```

- Parse from a string or .pl file
- Merge like maps
- No global state — REPL-friendly, test-friendly

Cuttable if pace is hot.

The knowledge base can change

```
(require '[talk.dynamic :as d]) ;; a mutable task tracker
```

```
(d/pending)      ;; => (buy-milk write-tests)
```

```
(d/complete! 'buy-milk)
```

```
(d/pending)      ;; => (write-tests)
```

```
(d/done)         ;; => (fix-bug buy-milk)
```

Same relational queries — but `assert/retract` mutate the world between them. No re-parse, no rebuild. *Cuttable if pace is hot.*

But what if the rules disagree?

We say: `alice has admin · admins delete · alice cannot delete.`

What does Prolog say?

→ to the REPL.

Live: who-can-do-what

→ terminal · bb repl · (require 'talk.relations)

(who-can 'delete) · (tu/solve {:rules rbac} '(can :user read)) · deny beat

(If this fails, we fall back to slide notes.)

Two questions, two solvers

Prolog answers: **who can?** (enumerate)

SMT answers: **is this consistent?** (decide)

You don't pick a *language*. You pick the solver that fits the *question*.

Same RBAC, in SMT-LIB 2

```
(declare-fun has-role (User Role) Bool)
(declare-fun can-delete (User) Bool)

;; admins may delete
(assert (! (= > (has-role alice admin) (can-delete alice))
          :named grant))

;; alice may not delete
(assert (! (not (can-delete alice))
          :named deny))

;; alice is an admin
(assert (! (has-role alice admin)
          :named assignment))

(check-sat)
```


The punchline

```
$ bb verify:example
× UNSAT — specification is unsatisfiable
  Conflicting assertions: grant, deny, assignment
```

Z3 doesn't just say *no*. It says **which three rules are fighting**.

That's an actionable error message your CI can paste into a PR comment.

Z3 decides. CLP solves.

```
(require '[spock.clp :as clp])

;; every positive (X,Y) with X + Y = 10 — describe it, don't loop it
(clp/solve "p(X,Y) :- X #> 0, Y #> 0, X + Y #= 10."
  "p(X,Y)" {:backend :clpz :labeling [:ff]})
;; => 9 answers: {:X 1 :Y 9} {:X 2 :Y 8} ... {:X 9 :Y 1}
```

Same *describe, don't traverse* shape — now over integer/boolean constraints. CLP($\mathbb{Z}$)/CLP(B) via Stryer Prolog. *Cuttable if pace is hot.*

Who else is calling `can-delete`?

```
$ clj -M:cli callers src/ --target myns/can-delete  
myns/can-delete  
  ← myns.api/handle-delete  
  ← myns.batch/cleanup  
  ← myns.admin/force-delete
```

Same engine — different question. *Where* does this rule get enforced?

100+ languages via tree-sitter. Clojure first-class via parser.

And while we're here — circular deps

```
$ clj -M:cli cycles src/  
Found 1 cycle:  
  myns.auth → myns.audit → myns.auth
```

`spock` self-analyses. The repo's `spock.svg` is what came out the day this slide was written.

Cuttable if pace is hot.

Your auth state machine, as a graph

```
stateDiagram-v2
    [*] --> Anon
    Anon --> Authenticated: login
    Authenticated --> Privileged: elevate
    Privileged --> Compromised: leak
    Authenticated --> Anon: logout
```

`spock_mermaid` parses this into Prolog facts (`state/1` + `transition/3`) and a `can_reach/2` rule.

Can a logged-in user be compromised?

```
$ clj -M:cli verify --input examples/auth.mmd \  
  --query "(can_reach authenticated compromised)" \  
  --explain
```

YES — derivation:

authenticated → privileged (elevate)
privileged → compromised (leak)

`.mmd` auto-converts to Prolog. The diagram becomes a model. The model becomes a proof obligation.

Same CLI. Three solvers.

```
clj -M:cli analyze src/ --entry myns/main      # graph
clj -M:cli cycles src/                          # graph
clj -M:cli verify --solver z3 --input p.smt2    # SMT
clj -M:cli verify --solver prolog --input r.pl  # Prolog (TuProlog)
clj -M:cli verify --solver scryer --input r.pl  # Prolog (Scryer, ISO-strict)
clj -M:cli verify --input g.mmd                # Mermaid → Prolog (auto)
clj -M:cli lint --solver z3 --input p.smt2
```

You don't switch tools. You switch *modes*.

What if an LLM could call all of this?

The CLI is one client. The MCP server is another.

Same engine. Different transport.

Spock as MCP — 9 tools

Tool	Domain
spock_verify	Z3 / Prolog formal verification
spock_lint	Static validation
spock_graph	Call-graph (Clojure + 100+ langs)
spock_mermaid	Diagram → Prolog
spock_skills	Search bundled templates
spock_formalize	NL → template + slot fill
spock_craft	Author a new template
spock_learn	Extract template from solution
spock_problog	Probabilistic queries

```
claude mcp add spock -- clj -M:spock-mcp
```

Claude verifies the RBAC mess

You: Check this RBAC policy for contradictions.

(Claude calls spock_verify with the SMT-LIB 2 source.)

Claude: UNSAT. The minimal conflicting subset is grant, deny, and assignment — alice is an admin, admins can delete, but alice is explicitly forbidden from deleting. Pick two of the three.

No hallucination — Z3 ran. Claude just read the structured result.

Structural fit, not hype

LLMs hallucinate. Solvers don't.

- **Deterministic** — same input, same answer.
- **Explainable** — `:explain true` returns the derivation trace.
- **Composable** — the result is data, not prose.

Prolog and SMT were *designed* for this shape of problem. We just didn't have a transport for it until 2024.

Three lenses, one mess

Lens	Question	Solver
Relations	who can?	Prolog
Conflicts	is this consistent?	Z3 / SMT
Reachability	can this happen?	Mermaid → Prolog

Pick the solver that fits the shape of the question.

Links

- **Spock** → github.com/burinc/b12n-spock
- **This deck** → github.com/burinc/b12n-slides-app/fixtures/fp-syd-spock-talk.json
- **FP Syd** → meetup.com/fp-syd

Also in spock 1.13: Stryer Prolog (ISO-strict), CLP($\mathbb{Z}$)/CLP(B) constraint solving, HTTP JSON API, ~50ms GraalVM native binary, 14-page user guide.

Questions?